AKSHAT H P.G
CB.SC.P2AI&C26004

Vanilla GCN

Phase 1: Basic understanding of vanilla GCN

- GCN (Graph Convolutional Network): a neural network that works directly on graph-structured, non-Euclidean data $G(V, E)$. Every layer updates each vertex's feature vector using the vertex itself and its neighbours, with one weight matrix W shared by all vertices.
- Vanilla GCN: the original form (Kipf and Welling, 2017): add self-loops, normalize the adjacency symmetrically by degree, multiply by W, apply ReLU. No attention, no neighbour sampling.
  - Input: adjacency matrix $A \in \{0,1\}^{N \times N}$, $N = |V|$ (number of vertices); feature matrix $H^{(0)} \in \mathbb{R}^{N \times F}$, one row per vertex.
  - Layer l: $H^{(l)} \in \mathbb{R}^{N \times F} \to H^{(l+1)}$. After L layers a vertex has seen everything within L hops.
  - Graph embedding: add up all rows of $H^{(L)}$ into one vector h_G. Two graphs are compared through their h_G.

Idea in one line: each layer is one round of "ask your neighbours, average their answers with yours, then re-weight".

Isomorphic graphs

$G_1 \cong G_2$ (G_1 is isomorphic to G_2) when there is a bijection $\phi : V_1 \to V_2$ with $(u, v) \in E_1 \leftrightarrow (\phi(u), \phi(v)) \in E_2$. In matrix form

$A_2 = P A_1 P^T$, with P a permutation matrix. The two graphs are the same structure with the vertices renamed.

Example: G_1 has edges $\{0–1, 0–2, 1–2, 2–3\}$. With $\Phi = \{0\to2, 1\to3, 2\to0, 3\to1\}$, G_2 has edges $\{2–3, 2–0, 3–0, 0–1\}$. So $G_1 \cong G_2$.

Injective, surjective, bijective (for $f : X \to Y$)

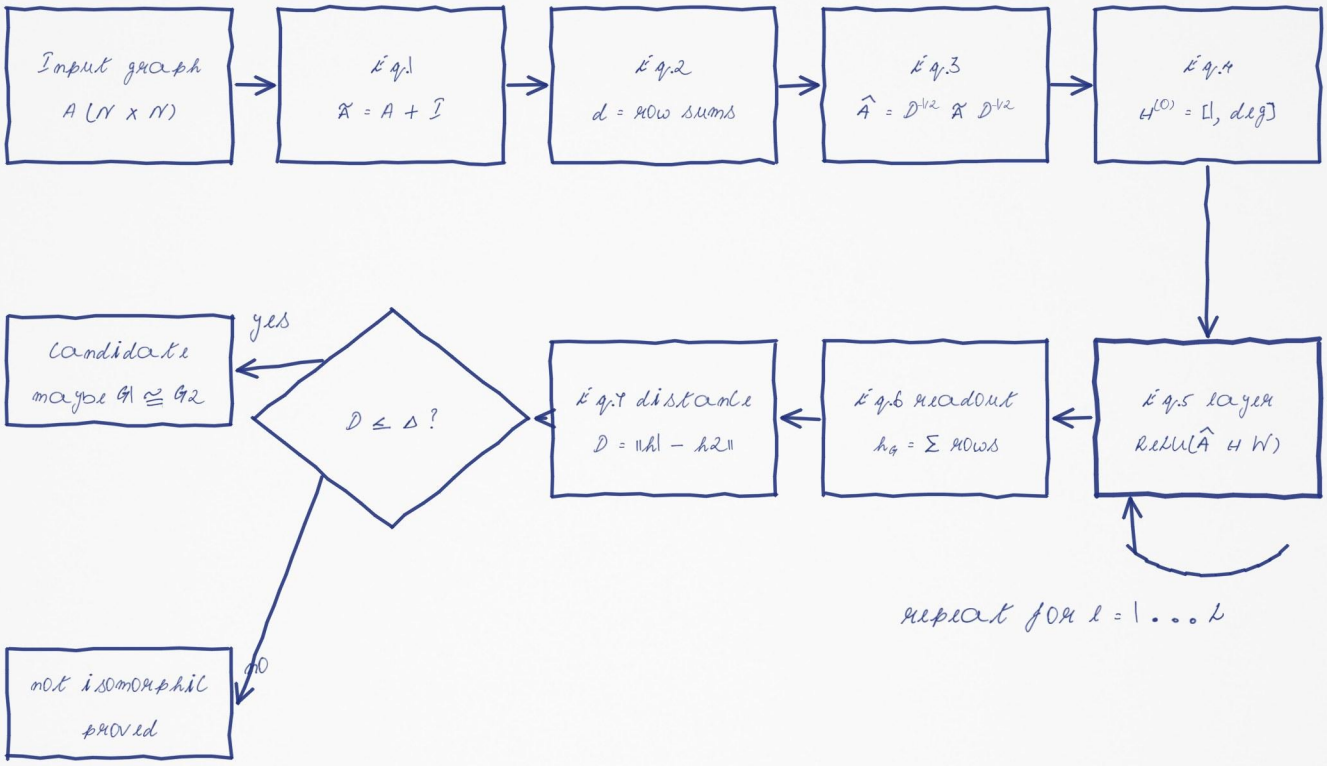
Property	Condition	Example
Injective (one-to-one)	$f(x_1) = f(x_2) \Rightarrow x_1 = x_2$	$f(x) = 2x$ on integers. no two inputs collide
Surjective (onto)	$\forall\, y \in Y\ \exists\, x \in X : f(x) = y$	$f(x) = x \bmod 3$ onto $\{0, 1, 2\}$
Bijective	injective and surjective	the vertex renaming Φ above

Why these matter for a GCN

1. Isomorphic graphs always get the same embedding. Renaming vertices only reorders the rows of $H^{(l)}$, and the sum h_G ignores order. So $G_1 \cong G_2 \Rightarrow h_{G1} = h_{G2}$, distance $D = 0$.
2. The converse fails. GCN is not injective on graphs. Mean-style aggregation merges different neighbourhoods. A 6-cycle C_6 and two separate triangles $2 \cdot C_3$ are not isomorphic, yet every vertex in both has degree 2, so both give the same h_G. This is the limit of the 1-WL (Weisfeiler–Lehman) test.
3. Reading the result. $D > 0$ proves the graphs are not isomorphic. $D = 0$ only says "isomorphism candidate". No polynomial-time algorithm is known for exact graph isomorphism, so the GCN gives a fast filter.
4. Surjectivity is not needed. Real outputs are never

negative, so the map onto R^E is not onto. For comparing graphs only injectivity matters.

Architecture diagram



vanilla GCN pipeline. Eq.1 to Eq.7 for two graphs each graph goes through the same pipeline with the same W; only the final distance decides. A distance above Δ proves the graphs differ, while a distance of zero only marks them as candidates.

Phase 2: Mathematical Concepts

Seven equations take a graph's adjacency matrix to one embedding vector h_G, and compare two graphs by the distance between their vectors. Every equation is worked out below for G_1 (edges 0–1, 0–2, 1–2, 2–3; $N = 4$).

Eq.1: self-loops.

Each vertex keeps its own features while aggregating.

$$\tilde{A}^{N \times N} = A^{N \times N} + I_N$$

Eq.2: degree vector with self-loops.

Multiply $\tilde{A}$ by the all-ones column 1_N.

$$\tilde{d}^{N \times 1} = \tilde{A}^{N \times N} \times 1^{N \times 1}, \qquad \tilde{d}_i = \sum_{j=1}^{N} \tilde{A}_{ij}$$

Ex: $\tilde{d} = [3, 3, 4, 2]$.

Eq.3: symmetric normalization.

Divides each entry by the square root of both end degrees, so high-degree vertices do not dominate.

$$\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$$

$$\hat{A}_{ij} = \frac{\tilde{A}_{ij}}{\sqrt{d_i d_j}}$$

$\hat{A}(G1)$	v0	v1	v2	v3
v0	0.3333	0.3333	0.2887	0
v1	0.3333	0.3333	0.2887	0
v2	0.2887	0.2887	0.2500	0.3536
v3	0	0	0.3536	0.5000

Example: $\hat{A}[0][2] = 1 / \sqrt{(3 \times 4)} = 0.2887$.

Eq.4: initial features.

The graphs have no attributes, so each vertex starts from its structure: a constant 1 and its degree (from A, without the self-loop).

$$H^{(0)}_i = [1, \deg(v_i)], \qquad H^{(0)} \in \mathbb{R}^{N \times 2}$$

G_1: $H^{(0)} = [1, 2], [1, 2], [1, 3], [1, 1]$.

Eq.5: vanilla GCN layer.

One weight matrix W, shared by all vertices and all layers, with $\mathrm{ReLU}(x) = \max(0, x)$.

$$H^{(l+1)} = \mathrm{ReLU}\left(\hat{A}\, H^{(l)} W\right)$$

$$W = \begin{bmatrix} 0.5 & -0.4 \\ 0.3 & 0.6 \end{bmatrix}, \qquad l = 0, 1, \ldots, L-1$$

Hand check for v0, layer 1: $H^{(0)}W$ gives $(1, 2)W = (1.1, 0.8)$, $(1, 3)W = (1.4, 1.4)$. Then $\hat{A}$ row v0 $= 0.3333 \cdot (1.1, 0.8) + 0.3333 \cdot (1.1, 0.8) + 0.2884 \cdot (1.4, 1.4) = (1.1345, 0.9345)$.

G_1	$H(1)$	$H(2)$
v0	(1.1345, 0.9345)	(0.8261, 0.0781)
v1	(1.1345, 0.9345)	(0.8261, 0.0781)
v2	(1.2679, 0.8826)	(0.9367, 0.0643)
v3	(0.8950, 0.5950)	(0.6307, 0.0074)

Eq.6: graph embedding (sum readout).

Adding the rows ignores vertex order, so the result does not depend on how the vertices are numbered.

$$h_G = \sum_{i=1}^{N} H^{(L)}_i = \mathbf{1}_N^{\top} H^{(L)}, \qquad h_G \in \mathbb{R}^{1 \times 2}$$

G_1: $h_{G1} = (0.8261 + 0.8261 + 0.9367 + 0.6307,\ 0.0781 + 0.0781 + 0.0643 + 0.0074) = (3.2197, 0.2310)$.

Eq.7: distance and decision.

$$D(G_1, G_2) = \| h_{G1} - h_{G2} \|_2 = \sqrt{\sum_j (h_{G1,j} - h_{G2,j})^2}$$

$D \le \Delta \Rightarrow$ isomorphism candidate, $\quad D > \Delta \Rightarrow G_1$ not $\cong G_2$

Bounds used: layers $L = 2$; tolerance $\Delta = 10^{-6}$; feature width 2; graph size $N \in [4, 6]$ in the test runs (the method works for any N).

Phase 3: Implementation Code (Java)

One method per equation; each method's comment names its equation. The program builds five graphs, runs Eq.1 to Eq.6 on each, then compares them with Eq.7.

Graph	Edges	What it tests
G_1	0–1, 0–2, 1–2, 2–3 (triangle with a tail)	the worked example of Phase 2
G_2	2–3, 2–0, 3–0, 0–1 (G_1 renamed by ϕ)	isomorphic to G_1, so D must be 0
G_3	0–1, 1–2, 2–3, 3–0 (4-cycle)	same N and number of edges as G_1, different shape
G_4	6-cycle C_6	non-isomorphic pair where the GCN fails
G_5	two triangles $2 \cdot C_3$	(G_4 vs G_5)

```
public class VanillaGCN
{
   // Eq.1 : Ãn = A + I
   static double[][] addSelfLoops(double[][] A)
   {
      int N = A.length;
      double[][] At = new double[N][N];
      for (int i = 0; i < N; i++)
         for (int j = 0; j < N; j++)
            At[i][j] = A[i][j] + (i == j ? 1 : 0);
      return At;
```

```
}

// Eq.2 : d~ = A~ x 1 (row sums)
static double[] degree(double[][] M)
{
    int N = M.length;
    double[] d = new double[N];
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            d[i] += M[i][j];
    return d;
}

// Eq.3 : A^[i][j] = A~[i][j] / sqrt(d~_i * d~_j)
static double[][] normalize(double[][] At, double[] dt)
{
    int N = At.length;
    double[][] Ah = new double[N][N];
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            Ah[i][j] = At[i][j] / Math.sqrt(dt[i] * dt[j]);
    return Ah;
}

// Eq.4 : H(0) row i = [1, deg(v_i)]
static double[][] initialFeatures(double[][] A)
{
    double[] d = degree(A);
    double[][] H = new double[d.length][2];
    for (int i = 0; i < d.length; i++) { H[i][0] = 1; H[i][1] = d[i]; }
    return H;
}

// Eq.5 : H(l+1) = ReLU( A^ x H(l) x W )
static double[][] layer(double[][] Ah, double[][] H, double[][] W)
{
    int N = H.length, f = H[0].length, g = W[0].length;
    double[][] HW = new double[N][g];
```

```
        for (int i = 0; i < n; i++)
            for (int k = 0; k < f; k++)
                for (int j = 0; j < g; j++)
                    HW[i][j] += H[i][k] * W[k][j];
        double[][] Hn = new double[n][g];
        for (int i = 0; i < n; i++)
            for (int k = 0; k < n; k++)
                if (Ah[i][k] != 0)
                    for (int j = 0; j < g; j++)
                        Hn[i][j] += Ah[i][k] * HW[k][j];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < g; j++)
                Hn[i][j] = Math.max(0, Hn[i][j]);
        return Hn;
    }

    // Eq.6 : h_G = sum over i of H(L)[i]
    static double[] readout(double[][] H)
    {
        double[] h = new double[H[0].length];
        for (double[] row : H)
            for (int j = 0; j < row.length; j++)
                h[j] += row[j];
        return h;
    }

    // Eq.7 : D(G1, G2) = || h_G1 - h_G2 ||
    static double distance(double[] h1, double[] h2)
    {
        double s = 0;
        for (int j = 0; j < h1.length; j++)
            s += (h1[j] - h2[j]) * (h1[j] - h2[j]);
        return Math.sqrt(s);
    }

    // full pipeline: Eq.1 -> Eq.6
    static double[] embed(String name, double[][] A, double[][] W, int L,
        boolean show)
```

```
{
    double[][] At = addSelfLoops(A);
    double[] dt = degree(At);
    double[][] Ah = normalize(At, dt);
    double[][] H = initialFeatures(A);
    for (int l = 1; l <= L; l++)
    {
        H = layer(Ah, H, W);
        if (show) { System.out.println("[Ex9.5] " + name + " H(" + l + "):");
            print(H); }
    }
    double[] h = readout(H);
    System.out.printf("[Ex9.6] h_%s = [%.4f, %.4f]%n", name, h[0], h[1]);
    return h;
}

static double[][] graph(int N, int[][] E)
{
    double[][] A = new double[N][N];
    for (int[] e : E) { A[e[0]][e[1]] = 1; A[e[1]][e[0]] = 1; }
    return A;
}

static void print(double[][] M)
{
    for (double[] row : M)
    {
        System.out.print(" [");
        for (double v : row) System.out.printf(" %.4f", v);
        System.out.println(" ]");
    }
}

static void compare(String a, double[] ha, String b, double[] hb,
    double delta)
{
    double D = distance(ha, hb);
    System.out.printf("[Ex9.7] D(%s,%s) = %.6f -> %s%n", a, b, D,
```

```
        D <= delta ? "same embedding (isomorphism candidate)" : "NOT
            isomorphic(");
    }

    public static void main(String[] args)
    {
        System.out.println("VANILLA GCN : GRAPH ISOMORPHISM CHECK");
        double[][] W = { {0.5, -0.4}, {0.3, 0.6} }; // shared weights
        int L = 2; // layers
        double delta = 1e-6; // tolerance

        double[][] G1 = graph(4, new int[][]{ {0,1}, {0,2}, {1,2}, {2,3} }); //
            triangle + tail
        double[][] G2 = graph(4, new int[][]{ {2,3}, {2,0}, {3,0}, {0,1} }); // G1
            relabelled
        double[][] G3 = graph(4, new int[][]{ {0,1}, {1,2}, {2,3}, {3,0} }); // 4-cycle
        double[][] G4 = graph(6, new int[][]{ {0,1}, {1,2}, {2,3}, {3,4}, {4,5}, {5,0}
            }); // C6
        double[][] G5 = graph(6, new int[][]{ {0,1}, {1,2}, {2,0}, {3,4}, {4,5}, {5,3}
            }); // 2 triangles

        double[] dt = degree(addSelfLoops(G1));
        System.out.printf("[Ex.2] d~(G1) = [%.0f, %.0f, %.0f, %.0f]%n",
            dt[0], dt[1], dt[2], dt[3]);
        System.out.println("[Ex.3] A^(G1):");
        print(normalize(addSelfLoops(G1), dt));
        System.out.println("[Ex.4] H(0)(G1):");
        print(initialFeatures(G1));

        double[] h1 = embed("G1", G1, W, L, true);
        double[] h2 = embed("G2", G2, W, L, false);
        double[] h3 = embed("G3", G3, W, L, false);
        double[] h4 = embed("G4", G4, W, L, false);
        double[] h5 = embed("G5", G5, W, L, false);

        compare("G1", h1, "G2", h2, delta);
        compare("G1", h1, "G3", h3, delta);
        compare("G4", h4, "G5", h5, delta);
    }
```

g

Output

```
VANILLA GCN : GRAPH ISOMORPHISM CHECK
[Eq.2] d~(G1) = [3, 3, 4, 2]
[Eq.3] Â(G1):
  [ 0.3333 0.3333 0.2887 0.0000 ]
  [ 0.3333 0.3333 0.2887 0.0000 ]
  [ 0.2887 0.2887 0.2500 0.3536 ]
  [ 0.0000 0.0000 0.3536 0.5000 ]
[Eq.4] H(0)(G1):
  [ 1.0000 2.0000 ]
  [ 1.0000 2.0000 ]
  [ 1.0000 3.0000 ]
  [ 1.0000 1.0000 ]
[Eq.5] G1 H(1):
  [ 1.1345 0.9345 ]
  [ 1.1345 0.9345 ]
  [ 1.2679 0.8826 ]
  [ 0.8950 0.5950 ]
[Eq.5] G1 H(2):
  [ 0.8261 0.0781 ]
  [ 0.8261 0.0781 ]
  [ 0.9367 0.0673 ]
  [ 0.6307 0.0074 ]
[Eq.6] h_G1 = [3.2197, 0.2310]
[Eq.6] h_G2 = [3.2197, 0.2310]
[Eq.6] h_G3 = [3.1600, 0.1600]
[Eq.6] h_G4 = [4.7400, 0.2400]
[Eq.6] h_G5 = [4.7400, 0.2400]
[Eq.7] D(G1,G2) = 0.000000 -> same embedding (isomorphism candidate)
[Eq.7] D(G1,G3) = 0.092725 -> NOT isomorphic
[Eq.7] D(G4,G5) = 0.000000 -> same embedding (isomorphism candidate)
```

Reading the output

- G1 vs G2: D = 0. Renaming the vertices does not change the embedding, as Phase I (point I) predicts.
- G1 vs G3: D = 0.0927 > Δ. Same number of vertices and edges,

but the 4-cycle's degrees are all 2, so the GCN separates them: proved not isomorphic.

- G4 vs G5: D = 0, but they are not isomorphic. Every vertex in both has degree 2, so every layer computes the same rows. This is the non-injectivity of Phase 1 (point 2), shown by running the code.
- In the G1 run printed above, ReLU did not change any value (all entries stayed positive). It stays in Eq.5 because with other weights or graphs it does.

Compile and run: javac VanillaGCN.java then java VanillaGCN.

Algorithm

The same steps as procedures, one per equation, in the notation of the class notes.

```
procedure AddSelfLoops(A, N) // Eq.1
// A[1..N][1..N] adjacency matrix
[
  for (i ← 1 ... N)
  [ for (j ← 1 ... N)
    [ if (i == j) Ã[i][j] ← A[i][j] + 1;
      else Ã[i][j] ← A[i][j];
    ] end for
  ] end for
  return Ã;
] end procedure

procedure Degree(M, N) // Eq.2
[
  for (i ← 1 ... N)
  [ d[i] ← 0;
    for (j ← 1 ... N) [ d[i] ← d[i] + M[i][j]; ] end for
  ] end for
  return d;
```

] end procedure

procedure normalize($\tilde{A}$, $\tilde{d}$, N) // Eq.3
[
  for (i ← 1 ... N)
  [for (j ← 1 ... N)
    [$\hat{A}$[i][j] ← $\tilde{A}$[i][j] / sqrt($\tilde{d}$[i] x $\tilde{d}$[j]);] end for
  ] end for
  return $\hat{A}$;
] end procedure

procedure InitialFeatures(A, N) // Eq.4
[
  call: d ← Degree(A, N);
  for (i ← 1 ... N) [H[i][1] ← 1; H[i][2] ← d[i];] end for
  return H;
] end procedure

procedure Layer($\hat{A}$, H, W, N, f, g) // Eq.5
// H[1:N][1:f], W[1:f][1:g]
[
  HW ← ∅; Hn ← ∅; // nullify
  for (i ← 1 ... N) // transform: HW = H x W
  [for (k ← 1 ... f)
    [for (j ← 1 ... g) [HW[i][j] ← HW[i][j] + H[i][k] x W[k][j];] end for
    ] end for
  ] end for
  for (i ← 1 ... N) // aggregate: Hn = $\hat{A}$ x HW
  [for (k ← 1 ... N)
    [if ($\hat{A}$[i][k] != 0)
      [for (j ← 1 ... g) [Hn[i][j] ← Hn[i][j] + $\hat{A}$[i][k] x HW[k][j];] end
        for
      ]
    ] end for
  ] end for
  for (i ← 1 ... N) // activate: ReLU
  [for (j ← 1 ... g) [Hn[i][j] ← max(0, Hn[i][j]);] end for
  ] end for

```
    return Hn;
] end procedure

procedure ReadOut(H, N, g) // Eq.6
[
    h ← ∅;
    for (i ← 1 ... N)
    [ for (j ← 1 ... g) [ h[j] ← h[j] + H[i][j]; ] end for
    ] end for
    return h;
] end procedure

procedure Compare(A1, A2, W, L, Δ) // Eq.7, using Eq.1 ... Eq.6
[
    for (each graph G ∈ {A1, A2})
    [ Call: Ã ← AddSelfLoops(A); Call: d̃ ← Degree(Ã);
      Call: Â ← Normalize(Ã, d̃); Call: H ← InitialFeatures(A);
      for (l ← 1 ... L) [ Call: H ← Layer(Â, H, W); ] end for
      Call: h_G ← ReadOut(H);
    ] end for
    D ← sqrt( Σ_j (h_G1[j] − h_G2[j])² );
    if (D <= Δ) return "isomorphism candidate";
    else return "NOT isomorphic";
] end procedure
```

Analysis

Comparing two graphs costs $O(L \cdot N^2)$ time and $O(N^2)$ space with the dense matrices used in the code. Here N = number of vertices, m = number of edges, L = number of layers, f and g = feature widths in and out of a layer ($f = g = 2$ here).

Procedure	Equation	Time $T(n)$	Space $S(n)$
AddSelfLoops	Eq.1	$O(N^2)$	$O(N^2)$
Degree	Eq.2	$O(N^2)$	$O(N)$
Normalize	Eq.3	$O(N^2)$	$O(N^2)$
InitialFeatures	Eq.4	$O(N^2)$ (calls Degree)	$O(N \cdot f)$
Layer (one call)	Eq.5	$O(N \cdot f \cdot g + N^2 + (N + 2m) \cdot g)$	$O(N \cdot g)$
Readout	Eq.6	$O(N \cdot g)$	$O(g)$
Distance	Eq.7	$O(g)$	$O(1)$

Layer: transform $H \times W \to N \cdot f \cdot g$

scan every $\hat{A}[i][k] \to N^2$

multiply the non-zeros $\to (N + 2m) \cdot g$ ($\hat{A}$ has N self-loops + 2 per edge)

ReLU $\to N \cdot g$

One graph = Eq.1 + Eq.2 + Eq.3 + Eq.4 + 2 x Eq.5 + Eq.6

$= O(N^2) + O(2 \cdot (N^2 + N \cdot f \cdot g))$

$= O(2 \cdot N^2) \to f, g$ constant

Two graphs + Eq.7 = 2 x $O(2 \cdot N^2) + O(g)$

Eq.$[T(n)]$: $O(2 \cdot N^2)$

Eq.$[S(n)]$: $O(N^2 + N \cdot g) \to \tilde{A}$ and $\hat{A}$ $(N \times N)$ + feature rows H

With adjacency lists instead of $N \times N$ matrices, only the non-zeros are stored and visited.

Eq.$[T(n)]$: $O(2 \cdot (N + m) \cdot g) \to$ linear in the size of the graph

Eq.$[S(n)]$: $O(N + m + N \cdot g)$

For sparse graphs (m much smaller than N^2) this is the version to use; the dense one is kept in the code because it

is easier to write out and to check by hand.